

بسم الله الرحمن الرحيم

دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده‌ی مهندسی کامپیوتر

پایان نامه‌ی کارشناسی

در رشته‌ی مهندسی کامپیوتر

طراحی و پیاده‌سازی سامانه‌ی واسط برای یکپارچه‌سازی سرویس‌های هوش مصنوعی

Design and Implementation of a Unified Backend Gateway
for AI Service Integration

ارائه‌دهنده: مصطفی رحمتی

شماره دانشجویی: 9931069

استاد راهنما: دکتر حسین زینلی

نیمسال دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

بهار ۱۴۰۵

تقديم به

[متن تقديم]

سپاسگزاری

[متن سپاسگزاری در اینجا نوشته می شود.]

چکیده

با گسترش روزافزون سرویس‌های هوش مصنوعی^۱ و تنوع رابط‌های برنامه‌نویسی کاربردی^۲ ارائه‌شده توسط شرکت‌های مختلف، سازمان‌ها و توسعه‌دهندگان با چالش یکپارچه‌سازی، مدیریت دسترسی و صورت‌حساب‌گذاری این سرویس‌ها روبه‌رو هستند. این پایان‌نامه طراحی و پیاده‌سازی یک سامانه‌ی واسطه^۳ یکپارچه برای دسترسی به سرویس‌های هوش مصنوعی را شرح می‌دهد.

سامانه‌ی پیاده‌سازی‌شده با استفاده از چارچوب Laravel (زبان PHP) در سمت سرور، پایگاه داده‌ی SQL Server، سیستم کش^۴ و صف^۵ مبتنی بر Redis، و رابط کاربری Vue.js طراحی شده است. این سامانه یک رابط برنامه‌نویسی سازگار با OpenAI ارائه می‌دهد که امکان اتصال به چهار ارائه‌دهنده‌ی هوش مصنوعی شامل OpenAI، Gemini، Groq و OpenRouter را فراهم می‌کند.

ویژگی‌های اصلی سامانه عبارت‌اند از: احراز هویت مبتنی بر رمز یک‌بارمصرف^۶، مدیریت کلیدهای رابط برنامه‌نویسی با محدودیت نرخ و لیست سفید آدرس IP، سیستم کیف پول و صدور صورت‌حساب، اندازه‌گیری مصرف توکن^۷ با استفاده از توکن‌ساز واقعی^۸، پشتیبانی از صورت‌حساب‌گذاری غیرتوکنی (تصویر، صوت)، ارسال گزارش^۹ به سرویس‌های خارجی از طریق صف، و گزارش‌دهی مصرف.

معماری سامانه بر الگوی خط لوله^{۱۰} استوار است که پردازش هر درخواست را به چهارده مرحله‌ی مستقل تقسیم می‌کند. همچنین از استراتژی کش‌کنار^{۱۱} برای کاهش بار پایگاه داده در داده‌های پرمصرف استفاده شده است.

نتایج ارزیابی نشان می‌دهد که سامانه با موفقیت درخواست‌های هوش مصنوعی را به ارائه‌دهندگان مختلف هدایت می‌کند، صورت‌حساب دقیق را بر اساس مصرف واقعی محاسبه می‌کند و از طریق مجموعه‌ی آزمون‌های واحد و ویژگی پوشش مناسبی دارد.

-
- AI Services^۱
 - Application Programming Interface (API)^۲
 - Gateway^۳
 - Cache^۴
 - Queue^۵
 - One-Time Password (OTP)^۶
 - Token^۷
 - Real Tokenizer^۸
 - Log^۹
 - Pipeline Pattern^{۱۰}
 - Cache-Aside Strategy^{۱۱}

کلیدواژه‌ها: سامانه‌ی واسط، هوش مصنوعی، رابط برنامه‌نویسی یکپارچه، الگوی خط
لوله، مدیریت سرویس، صورت‌حساب، Laravel، Redis، OpenAI

فهرست مطالب

ث	سپاسگزاری	
چ	چکیده	
۱	۱ مقدمه	
۱	۱-۱ کلیات	
۲	۲-۱ بیان مسئله	
۳	۳-۱ انگیزه و ضرورت	
۳	۴-۱ اهداف پروژه	
۴	۵-۱ دستاوردهای پروژه	
۵	۶-۱ تغییرات نسبت به پروپوزال	
۵	۷-۱ ساختار پایان نامه	
۶	۲ مروری بر پیشینه	
۶	۱-۲ مقدمه	
۶	۲-۲ مفاهیم پایه	
۶	۱-۲-۲ سامانه‌ی واسط API	
۶	۲-۲-۲ معماری خدمات‌گرا	
۶	۳-۲-۲ مدل‌های زبانی بزرگ	
۶	۳-۲ سیستم‌های مشابه	
۶	۱-۳-۲ Groq	
۶	۲-۳-۲ OpenRouter	
۶	۳-۳-۲ Anyscale Endpoints	
۶	۴-۳-۲ OpenAI	
۶	۴-۲ مقایسه و جمع‌بندی	
۷	۳ طراحی سیستم	
۸	۱-۳ مقدمه	
۸	۲-۳ معماری کلی سیستم	
۸	۱-۲-۳ نمودار اجزای سیستم	

۸	۲-۲-۳ جریان درخواست
۸	۳-۳ مدل داده
۸	۱-۳-۳ جداول احراز هویت و دسترسی
۸	۲-۳-۳ جداول کلیدهای API
۸	۳-۳-۳ جداول ارائه‌دهندگان و مدل‌ها
۸	۴-۳-۳ جداول صورت‌حساب
۸	۵-۳-۳ جداول گزارش مصرف
۸	۴-۳ طراحی API
۸	۵-۳ الگوی خط لوله
۸	۶-۳ استراتژی کش
۸	۷-۳ طراحی سیستم صورت‌حساب

۴ پیاده‌سازی

۹	۱-۴ مقدمه
۹	۲-۴ فناوری‌های استفاده‌شده
۹	۱-۲-۴ چارچوب Laravel
۹	۲-۲-۴ پایگاه داده‌ی SQL Server
۹	۳-۲-۴ Redis
۱۰	۴-۲-۴ Vue.js
۱۰	۳-۴ ساختار پوشه‌بندی پروژه
۱۰	۴-۴ پیاده‌سازی خط لوله‌ی سامانه‌ی واسط
۱۱	۵-۴ پیاده‌سازی آداپتورهای ارائه‌دهندگان
۱۲	۱-۵-۴ OpenAI
۱۲	۲-۵-۴ Gemini
۱۲	۳-۵-۴ OpenRouter و Groq
۱۳	۶-۴ پیاده‌سازی احراز هویت OTP
۱۵	۷-۴ پیاده‌سازی مدیریت کلیدهای API
۱۷	۸-۴ پیاده‌سازی سیستم صورت‌حساب
۱۹	۹-۴ پیاده‌سازی گزارش‌دهی و ارسال گزارش خارجی
۲۰	۱۰-۴ داشبورد Playground

۵ ارزیابی

۲۱	۱-۵ مقدمه
۲۱	۲-۵ ارزیابی فنی
۲۱	۱-۲-۵ آزمون‌های واحد
۲۱	۲-۲-۵ آزمون‌های ویژگی
۲۱	۳-۲-۵ نتایج آزمون
۲۱	۳-۵ ارزیابی عملکرد
۲۱	۱-۳-۵ تأخیر پاسخ

۲۱	۲-۳-۵ دقت صورت حساب گذاری
۲۱	۴-۵ مقایسه با اهداف
۲۲	۶ نتیجه گیری
۲۲	۱-۶ جمع بندی
۲۲	۲-۶ دستاوردها
۲۲	۳-۶ محدودیت ها
۲۲	۴-۶ کارهای آتی
۲۴	مستندات API
۲۴	۱- نقاط پایانی احراز هویت
۲۴	۲- نقاط پایانی کاربر
۲۵	۳- نقاط پایانی کلیدهای API
۲۵	۴- نقاط پایانی هوش مصنوعی
۲۶	۵- نقاط پایانی کیف پول و اشتراک
۲۶	۶- نقاط پایانی گزارش ها

فهرست تصاویر

۱۱	۱-۴	فراخوانی نقطه‌ی پایانی تکمیل گفتگو از طریق سامانه‌ی واسط
۱۲	۲-۴	ثبت ارائه‌دهنده‌ی Groq در سامانه
۱۳	۳-۴	ثبت مدل llama-3.3-70b-versatile برای ارائه‌دهنده‌ی Groq
۱۴	۴-۴	ارسال درخواست دریافت رمز یک‌بارمصرف
۱۴	۵-۴	تأیید رمز یک‌بارمصرف و دریافت توکن احراز هویت
۱۵	۶-۴	دریافت اطلاعات پروفایل کاربر پس از احراز هویت
۱۶	۷-۴	ایجاد کلاینت API برای یک کاربر
۱۶	۸-۴	ایجاد کلید API و دریافت توکن دسترسی
۱۷	۹-۴	شارژ کیف پول کاربری
۱۸	۱۰-۴	ایجاد طرح اشتراک
۱۹	۱۱-۴	خرید اشتراک توسط کاربر
۲۰	۱۲-۴	گزارش مصرف کاربر به تفکیک درخواست

فهرست جداول

فصل ۱

مقدمه

۱-۱ کلیات

در دهه‌ی اخیر، هوش مصنوعی^۱ تحولی بنیادین در حوزه‌های مختلف فناوری اطلاعات ایجاد کرده است. ظهور مدل‌های زبانی بزرگ^۲ مانند GPT-4o، Gemini و Claude، امکانات بی‌سابقه‌ای را برای توسعه‌دهندگان نرم‌افزار و سازمان‌ها فراهم کرده است. این مدل‌ها از طریق رابط‌های برنامه‌نویسی کاربردی^۳ در دسترس عموم قرار گرفته‌اند و کاربردهای گسترده‌ای در حوزه‌هایی از جمله دستیاران هوشمند، تولید محتوا، تحلیل داده، تبدیل گفتار به متن و تولید تصویر یافته‌اند [۱].

شرکت‌های بزرگ فناوری از جمله Google، OpenAI، Anthropic (با محصول Gemini)، Groq و سرویس‌های واسط مانند OpenRouter هر کدام رابط‌های برنامه‌نویسی منحصربه‌فرد خود را ارائه می‌دهند. گرچه بسیاری از این رابط‌ها قابلیت‌های مشابهی دارند، اما در ساختار درخواست، احراز هویت، فرمت پاسخ و سیستم صورت‌حساب‌گذاری تفاوت‌های قابل توجهی دارند [۲، ۳].

این تنوع باعث شده که سازمان‌هایی که قصد دارند از چند سرویس هوش مصنوعی به طور هم‌زمان بهره ببرند یا از یک ارائه‌دهنده به ارائه‌دهنده‌ی دیگری مهاجرت کنند، با چالش‌های فنی و مدیریتی قابل توجهی روبه‌رو شوند. در این شرایط، وجود یک سامانه‌ی واسط^۴ که بتواند ارتباط با سرویس‌های مختلف را به صورت استاندارد مدیریت کند، نیاز ملموسی است.

این پروژه با هدف طراحی و پیاده‌سازی چنین سامانه‌ای تعریف شده است. سامانه‌ی پیاده‌سازی‌شده به عنوان یک لایه‌ی میانجی بین سرویس‌گیرندگان^۵ و ارائه‌دهندگان مختلف هوش مصنوعی عمل می‌کند و یک رابط برنامه‌نویسی استاندارد و یکپارچه را در اختیار توسعه‌دهندگان قرار می‌دهد. علاوه بر این، قابلیت‌هایی مانند احراز هویت،

Artificial Intelligence (AI)^۱

Large Language Models (LLM)^۲

Application Programming Interface (API)^۳

Gateway^۴

Clients^۵

مدیریت کلیدهای رابط برنامه‌نویسی، کیف پول، خرید اشتراک^۶، صدور فاکتور و گزارش‌های مصرف نیز در این سامانه پیش‌بینی شده است.

۲-۱ بیان مسئله

در اکوسیستم کنونی هوش مصنوعی، هر ارائه‌دهنده‌ی سرویس رابط برنامه‌نویسی مخصوص خود را تعریف می‌کند. برای مثال، OpenAI یک ساختار مشخص برای ارسال پیام و دریافت پاسخ دارد، Gemini از API بومی Google با فرمت متفاوتی استفاده می‌کند، و Groq اگرچه سازگار با فرمت OpenAI است، اما API Key و نقاط پایانی^۷ جداگانه‌ای دارد. این تفاوت‌ها در چند سطح مشکل‌ساز هستند:

۱. **چندگانگی پیاده‌سازی:** توسعه‌دهنده‌ای که می‌خواهد از چند سرویس هوش مصنوعی استفاده کند، ملزم است برای هر کدام کد جداگانه‌ای بنویسد. این موضوع حجم کد را افزایش داده و نگهداری سیستم را دشوار می‌کند.

۲. **چالش مهاجرت:** در صورتی که سازمانی بخواهد از یک ارائه‌دهنده به ارائه‌دهنده‌ی دیگری مهاجرت کند—برای مثال از OpenAI به Groq—ناچار به تغییرات گسترده در کدپایه^۸ خواهد بود.

۳. **پراکندگی مدیریت:** کلیدهای API، محدودیت نرخ^۹، گزارش مصرف و صورت‌حساب‌ها برای هر سرویس به صورت مجزا مدیریت می‌شوند. این پراکندگی دید جامعی از مصرف کل را از کاربر می‌گیرد.

۴. **کمبود ابزار مدیریت مالی:** اکثر ارائه‌دهندگان سرویس هوش مصنوعی امکانات محدودی برای مدیریت مصرف چندکاربره، کیف پول داخلی، صدور فاکتور یا خرید اشتراک دارند. این نیاز برای سازمان‌هایی که می‌خواهند دسترسی به هوش مصنوعی را برای کاربران داخلی خود مدیریت کنند، بیشتر احساس می‌شود.

۵. **نبود کنترل و نظارت متمرکز:** بدون یک سامانه‌ی واسطه، کنترل دسترسی، نظارت بر مصرف، تعیین سقف مصرف و ثبت گزارش‌های تفصیلی برای هر ارائه‌دهنده باید به صورت جداگانه پیاده‌سازی شود.

این مشکلات با گسترش استفاده از هوش مصنوعی در سازمان‌ها حادتر می‌شود. راهکار پیشنهادی این پروژه، طراحی و پیاده‌سازی یک سامانه‌ی واسطه است که تمامی این چالش‌ها را از طریق یک لایه‌ی مرکزی و یکپارچه حل کند.

^۶ Subscription

^۷ Endpoint

^۸ Codebase

^۹ Rate Limit

۳-۱ انگیزه و ضرورت

انگیزه‌ی اصلی این پروژه از نیاز واقعی سازمان‌ها و توسعه‌دهندگانی نشأت می‌گیرد که می‌خواهند از قابلیت‌های هوش مصنوعی بهره ببرند اما نمی‌خواهند درگیر پیچیدگی‌های یکپارچه‌سازی چندین سرویس مجزا شوند. ضرورت چنین سامانه‌ای را می‌توان از چند منظر بررسی کرد:

از منظر فنی، با داشتن یک رابط برنامه‌نویسی استاندارد و سازگار با OpenAI—که پرکاربردترین فرمت در این حوزه است—توسعه‌دهندگان می‌توانند بدون تغییر در کد سمت مشتری، از ارائه‌دهندگان مختلف استفاده کنند. تنها با تغییر فیلدهای provider و model در درخواست، سامانه درخواست را به ارائه‌دهنده‌ی مناسب هدایت می‌کند.

از منظر مالی، یک سیستم متمرکز مدیریت کیف پول و صورت‌حساب، امکان صدور فاکتور یکپارچه، ردیابی دقیق مصرف و کنترل هزینه‌ها را فراهم می‌کند. این موضوع برای سازمان‌هایی که هزینه‌های هوش مصنوعی بخش قابل توجهی از بودجه‌ی عملیاتی آن‌ها را تشکیل می‌دهد، از اهمیت ویژه‌ای برخوردار است.

از منظر امنیتی، متمرکزسازی احراز هویت و مدیریت کلیدهای API در یک سامانه‌ی واحد، امکان اعمال سیاست‌های امنیتی منسجم—مانند محدودیت نرخ درخواست، لیست سفید آدرس IP و ممیزی^{۱۰} عملیات حساس—را به صورت مرکزی فراهم می‌آورد.

از منظر مقیاس‌پذیری، الگوی خط لوله^{۱۱} به‌کاررفته در این سامانه، امکان افزودن مراحل جدید به پردازش درخواست—مثلاً مدار قطعه^{۱۲} یا راهبری هوشمند^{۱۳}—را بدون تغییر در ساختار اصلی فراهم می‌کند.

۴-۱ اهداف پروژه

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک سامانه‌ی واسط جامع برای یکپارچه‌سازی سرویس‌های هوش مصنوعی است. این هدف کلی به اهداف عملیاتی زیر تجزیه می‌شود:

۱. **ارائه‌ی رابط برنامه‌نویسی یکپارچه:** پیاده‌سازی مجموعه‌ای از نقاط پایانی^{۱۴} سازگار با استاندارد OpenAI، شامل: تکمیل گفتگو^{۱۵}، جاسازی متن^{۱۶}، تولید تصویر^{۱۷}، تبدیل گفتار به متن^{۱۸} و تبدیل متن به گفتار^{۱۹}.

^{۱۰}Audit

^{۱۱}Pipeline Pattern

^{۱۲}Circuit Breaker

^{۱۳}Smart Routing

^{۱۴}Endpoints

^{۱۵}Chat Completions

^{۱۶}Embeddings

^{۱۷}Image Generation

^{۱۸}Audio Transcription

^{۱۹}Text-to-Speech

۲. **پشتیبانی از چند ارائه‌دهنده:** پیاده‌سازی آداپتور^{۲۰} برای ارائه‌دهندگان OpenAI، Gemini، Groq و OpenRouter با امکان افزودن ارائه‌دهندگان جدید به صورت اتصال‌پذیر^{۲۱}.
۳. **احراز هویت و مدیریت دسترسی:** پیاده‌سازی جریان احراز هویت مبتنی بر رمز یک‌بارمصرف برای ورود به داشبورد مدیریتی، و مدیریت کلیدهای API برای درخواست‌های برنامه‌ای.
۴. **سیستم صورت‌حساب:** پیاده‌سازی کیف پول کاربری، ثبت دقیق مصرف توکن و منابع غیرتوکنی، صدور فاکتور و مدیریت اشتراک‌های ماهانه.
۵. **گزارش‌دهی و مانیتورینگ:** فراهم کردن گزارش‌های تفصیلی مصرف، دفترکل تراکنش‌های کیف پول و لیست فاکتورها، همراه با ارسال گزارش‌های خط لوله به سرویس‌های مدیریت گزارش خارجی.
۶. **آزمون و مستندسازی:** نوشتن مجموعه‌ی آزمون‌های واحد و ویژگی برای تمامی ماژول‌ها، و تولید مستندات OpenAPI با استفاده از ابزار Swagger.

۵-۱ دستاوردهای پروژه

پروژه‌ی حاضر دستاوردهای زیر را به عنوان خروجی اصلی ارائه می‌دهد:

- **سامانه‌ی واسط عملیاتی:** یک نرم‌افزار سمت سرور کاملاً عملیاتی که درخواست‌های هوش مصنوعی را از سرویس‌گیرندگان دریافت و به چهار ارائه‌دهنده‌ی OpenAI، Gemini، Groq و OpenRouter هدایت می‌کند.
- **رابط برنامه‌نویسی سازگار با OpenAI:** پیاده‌سازی پنج نقطه‌ی پایانی هوش مصنوعی با فرمت سازگار با استاندارد OpenAI، به گونه‌ای که سرویس‌گیرندگان موجود با حداقل تغییر می‌توانند به این سامانه متصل شوند.
- **معماری مبتنی بر خط لوله:** پیاده‌سازی چهارده مرحله‌ی پردازشی در قالب الگوی خط لوله‌ی Laravel، که هر مرحله مسئولیتی مشخص و مستقل دارد.
- **سیستم صورت‌حساب دقیق:** اندازه‌گیری مصرف توکن با استفاده از توکن‌ساز واقعی BPE، پشتیبانی از صورت‌حساب‌گذاری منابع غیرتوکنی (تصویر، صوت) و مدیریت کیف پول با تراکنش‌های ایمن.
- **داشبورد Playground:** یک رابط کاربری Vue.js برای آزمایش تعاملی نقاط پایانی سامانه، با امکان انتخاب ارائه‌دهنده، مدل، کلید API و مشاهده‌ی پاسخ.
- **پوشش آزمون:** مجموعه‌ای از آزمون‌های واحد و ویژگی که جریان‌های اصلی سامانه را پوشش می‌دهند.

^{۲۰}Adapter
^{۲۱}Pluggable

۶-۱ تغییرات نسبت به پروپوزال

در پروپوزال^{۲۲} اولیه‌ی این پروژه، بسته‌ی فناوری پیش‌بینی‌شده شامل زبان Python، چارچوب‌های FastAPI یا Django، و پایگاه داده‌ی PostgreSQL بود. در مرحله‌ی اجرا، این بسته‌ی فناوری به دلایل فنی و عملی تغییر یافت و پیاده‌سازی نهایی با فناوری‌های زیر انجام شد:

- چارچوب Laravel (زبان PHP) – به عنوان چارچوب اصلی سمت سرور^{۲۳}.
- پایگاه داده‌ی SQL Server – به عنوان منبع اصلی ذخیره‌ی داده.
- Redis – به عنوان موتور کش^{۲۴} و مدیریت صف‌های پردازش پس‌زمینه.
- Vue.js – به عنوان چارچوب رابط کاربری برای میدان بازی^{۲۵}.

اهداف اصلی پروژه – شامل یکپارچه‌سازی چند ارائه‌دهنده‌ی هوش مصنوعی، سیستم احراز هویت، مدیریت کلیدهای API، کیف پول، اشتراک و گزارش‌دهی – بدون تغییر باقی ماندند و کاملاً پیاده‌سازی شدند.

۷-۱ ساختار پایان‌نامه

مابقی این پایان‌نامه به صورت زیر سازماندهی شده است:

فصل دوم – مروری بر پیشینه: ابتدا مفاهیم پایه‌ی سامانه‌های واسط و رابط‌های برنامه‌نویسی

هوش مصنوعی بررسی می‌شوند. سپس سیستم‌های مشابه از جمله Groq، OpenRouter، Anyscale Endpoints و OpenAI تحلیل و مقایسه می‌شوند.

فصل سوم – طراحی سیستم: معماری کلی سامانه، مدل داده، طراحی API، الگوی خط لوله، استراتژی کش و طراحی سیستم صورت‌حساب تشریح می‌شوند.

فصل چهارم – پیاده‌سازی: جزئیات پیاده‌سازی هر ماژول از جمله خط لوله‌ی سامانه‌ی واسط، آداپتورهای ارائه‌دهندگان، احراز هویت، مدیریت کلیدهای API و سیستم صورت‌حساب ارائه می‌شود.

فصل پنجم – ارزیابی: نتایج آزمون‌های واحد و ویژگی، ارزیابی عملکرد و مقایسه با اهداف اولیه ارائه می‌شوند.

فصل ششم – نتیجه‌گیری: دستاوردهای کلی پروژه جمع‌بندی شده و پیشنهادهایی برای کارهای آتی ارائه می‌شود.

^{۲۲} Proposal

^{۲۳} Server-side

^{۲۴} Cache Engine

^{۲۵} Playground

فصل ۲

مروری بر پیشینه

۱-۲ مقدمه

۲-۲ مفاهیم پایه

۱-۲-۲ سامانه‌ی واسط API

۲-۲-۲ معماری خدمات‌گرا

۳-۲-۲ مدل‌های زبانی بزرگ

۳-۲ سیستم‌های مشابه

۱-۳-۲ Groq

۲-۳-۲ OpenRouter

۳-۳-۲ Anyscale Endpoints

۴-۳-۲ OpenAI

۴-۲ مقایسه و جمع‌بندی

فصل ۳

طراحی سیستم

۱-۳ مقدمه

۲-۳ معماری کلی سیستم

۱-۲-۳ نمودار اجزای سیستم

۲-۲-۳ جریان درخواست

۳-۳ مدل داده

۱-۳-۳ جداول احراز هویت و دسترسی

۲-۳-۳ جداول کلیدهای API

۳-۳-۳ جداول ارائه‌دهندگان و مدل‌ها

۴-۳-۳ جداول صورت‌حساب

۵-۳-۳ جداول گزارش مصرف

۴-۳ طراحی API

۵-۳ الگوی خط لوله

۶-۳ استراتژی کش

۷-۳ طراحی سیستم صورت‌حساب

فصل ۴

پیاده‌سازی

۴-۱ مقدمه

در این فصل جزئیات پیاده‌سازی سامانه‌ی واسط هوش مصنوعی تشریح می‌شود. هر ماژول به ترتیب وابستگی معرفی شده و برای هر بخش نمونه‌های عملی از فراخوانی API ارائه می‌گردد. تمامی نمونه‌ها با ابزار Postman بر روی نمونه‌ی محلی سامانه اجرا شده‌اند.

۴-۲ فناوری‌های استفاده‌شده

۴-۲-۱ چارچوب Laravel

Laravel یک چارچوب PHP با معماری MVC است که زیرساخت قدرتمندی برای ساخت APIهای RESTful فراهم می‌کند. استفاده از الگوی Pipeline داخلی Laravel امکان پیاده‌سازی خط لوله‌ی پردازش درخواست را با اتصال‌پذیری بالا فراهم کرده است.

۴-۲-۲ پایگاه داده‌ی SQL Server

Microsoft SQL Server به عنوان پایگاه داده‌ی اصلی انتخاب شده است. تمامی اطلاعات کاربران، کلیدهای API، ارائه‌دهندگان، گزارش‌های مصرف و تراکنش‌های مالی در این پایگاه داده ذخیره می‌شوند.

۴-۲-۳ Redis

Redis برای دو منظور به کار می‌رود: کش کردن نتایج پرتکرار (مانند پیکربندی ارائه‌دهندگان و مدل‌ها) و مدیریت صف‌های پردازش پس‌زمینه برای ارسال رمز یک‌بارمصرف.

۴-۲-۴ Vue.js

Vue.js برای پیاده‌سازی داشبورد Playground استفاده شده است که امکان آزمایش تعاملی نقاط پایانی سامانه را فراهم می‌کند.

۳-۴ ساختار پوشه‌بندی پروژه

پروژه از الگوی معماری حوزه‌محور^۱ پیروی می‌کند. کلاس‌های مربوط به هر حوزه‌ی کاری در پوشه‌ی app/Domains قرار دارند که شامل زیرحوزه‌های احراز هویت (Auth)، سامانه‌ی واسط (Gateway)، کلیدهای API (Keys) و ارائه‌دهندگان (Providers) است. کنترلرها در app/Http/Controllers/Api/V1 و خط لوله‌های پردازش درخواست در app/Pipelines نگهداری می‌شوند.

۴-۴ پیاده‌سازی خط لوله‌ی سامانه‌ی واسط

هسته‌ی سامانه یک خط لوله‌ی چهارده‌مرحله‌ای است که هر درخواست AI را از لحظه‌ی ورود تا تحویل پاسخ پردازش می‌کند:

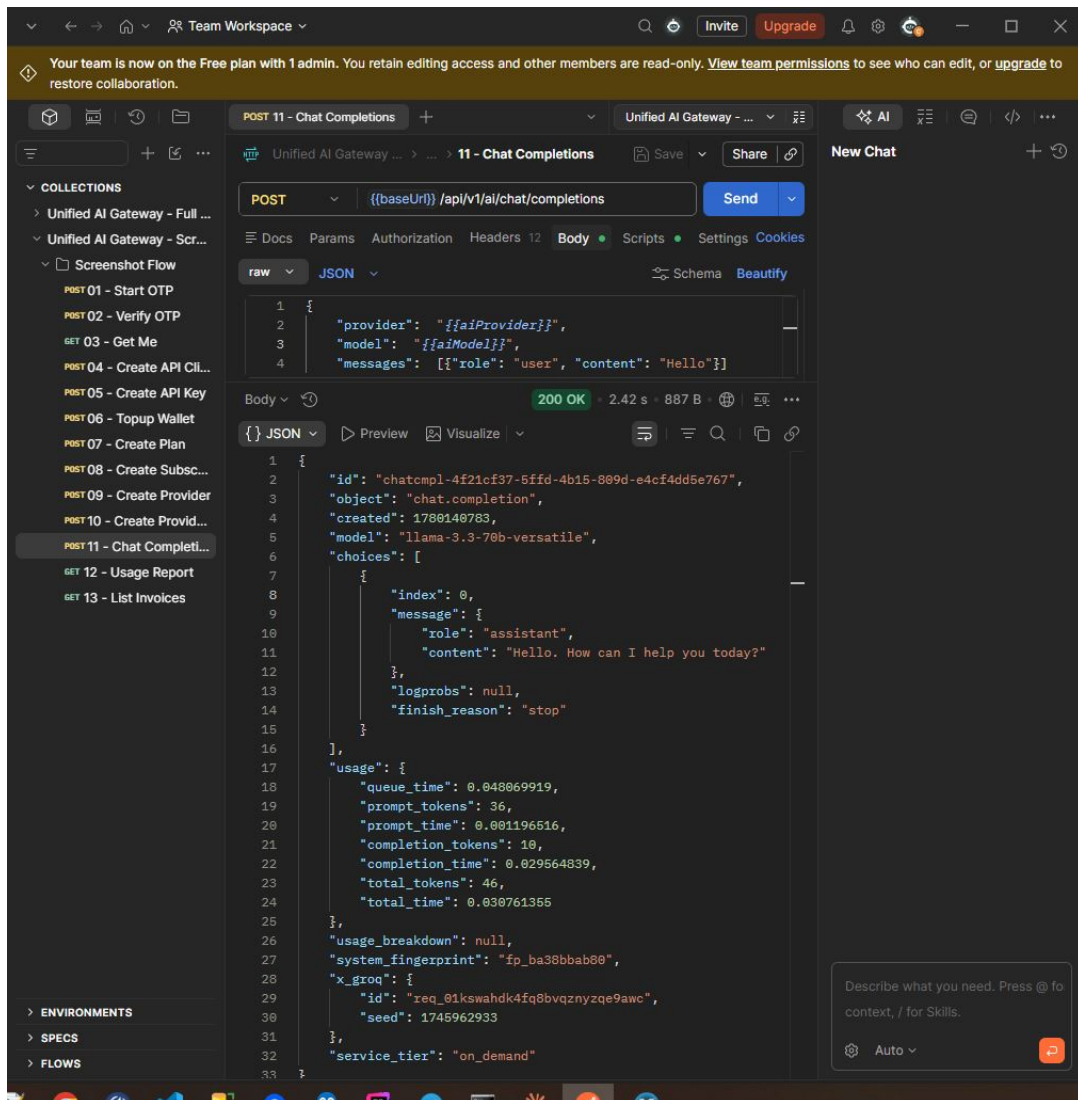
۱. احراز هویت کلید API – تأیید هویت سرویس‌گیرنده با HMAC-SHA256
۲. اعمال لیست سفید IP – بررسی آدرس IP در صورت تعریف
۳. اعتبارسنجی درخواست – بررسی فیلدهای اجباری و فرمت
۴. محدودیت نرخ – کنترل تعداد درخواست در دقیقه
۵. انتخاب ارائه‌دهنده – بارگذاری پیکربندی ارائه‌دهنده از DB/کش
۶. اعتبارسنجی انتخاب ارائه‌دهنده – تأیید وجود ارائه‌دهنده و مدل در پایگاه داده
۷. تخمین مصرف – برآورد هزینه پیش از ارسال
۸. بررسی کیف پول یا اشتراک – اطمینان از کافی بودن موجودی
۹. ارسال درخواست به ارائه‌دهنده – فراخوانی HTTP از طریق آداپتور
۱۰. نرمال‌سازی پاسخ – تبدیل پاسخ به فرمت یکپارچه
۱۱. اندازه‌گیری مصرف واقعی – شمارش توکن با توکن‌ساز BPE
۱۲. ثبت گزارش – درج رکورد مصرف در پایگاه داده

^۱Domain-Driven Design

۱۳. کسر از کیف پول – تسویه‌ی هزینه‌ی واقعی

۱۴. ارسال گزارش خارجی – ارسال به BetterStack/Loki در صورت پیکربندی

شکل ۴-۱ نمونه‌ی کامل یک فراخوانی موفق تکمیل گفتگو را نشان می‌دهد که از تمام مراحل این خط لوله عبور کرده است.



شکل ۴-۱: فراخوانی نقطه‌ی پایانی تکمیل گفتگو از طریق سامانه‌ی واسط

۵-۴ پیاده‌سازی آداپتورهای ارائه‌دهندگان

برای هر ارائه‌دهنده یک آداپتور جداگانه پیاده‌سازی شده که رابط `ProviderAdapterInterface` را پیاده‌سازی می‌کند.

OpenAI ۱-۵-۴

آداپتور OpenAI به صورت مستقیم با نقاط پایانی استاندارد این شرکت ارتباط برقرار می‌کند و پشتیبانی کاملی از تمام نقاط پایانی پیاده‌سازی شده دارد.

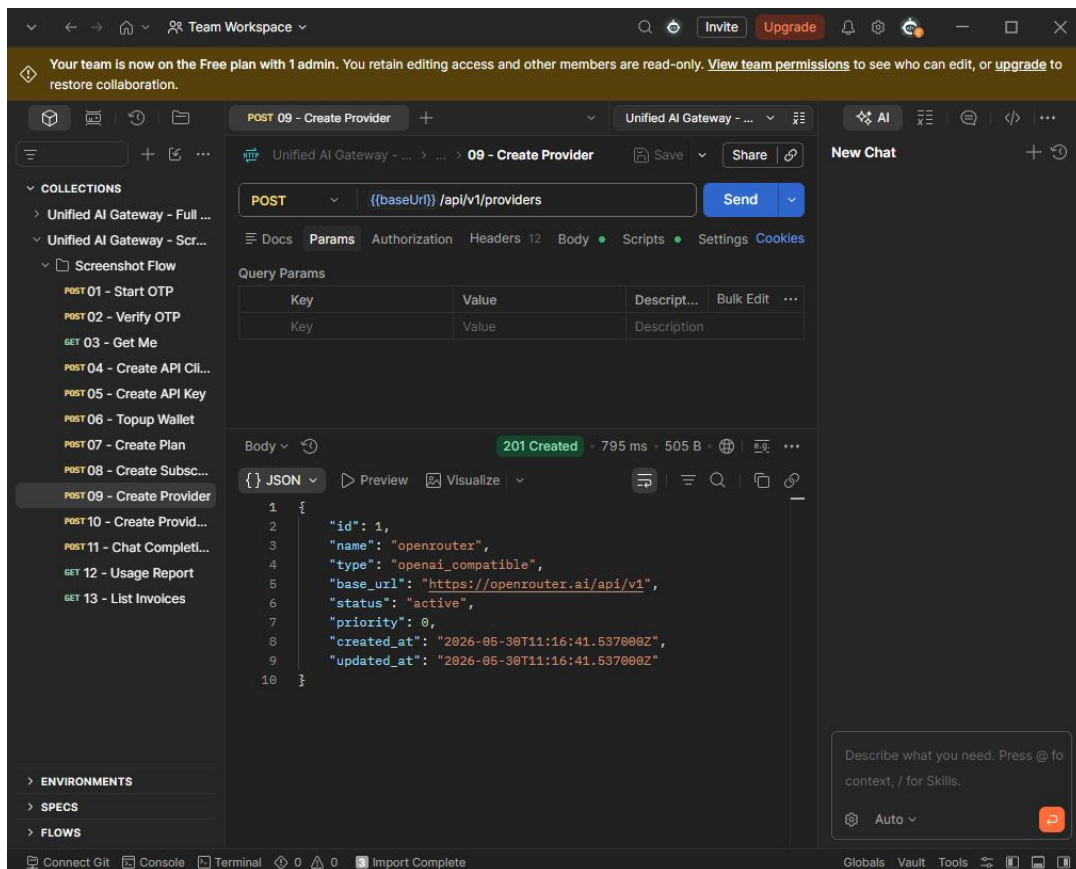
Gemini ۲-۵-۴

آداپتور Gemini درخواست‌های با فرمت OpenAI را به فرمت بومی Google Generative AI API تبدیل کرده و پاسخ را به فرمت استاندارد نرمال می‌کند.

OpenRouter و Groq ۳-۵-۴

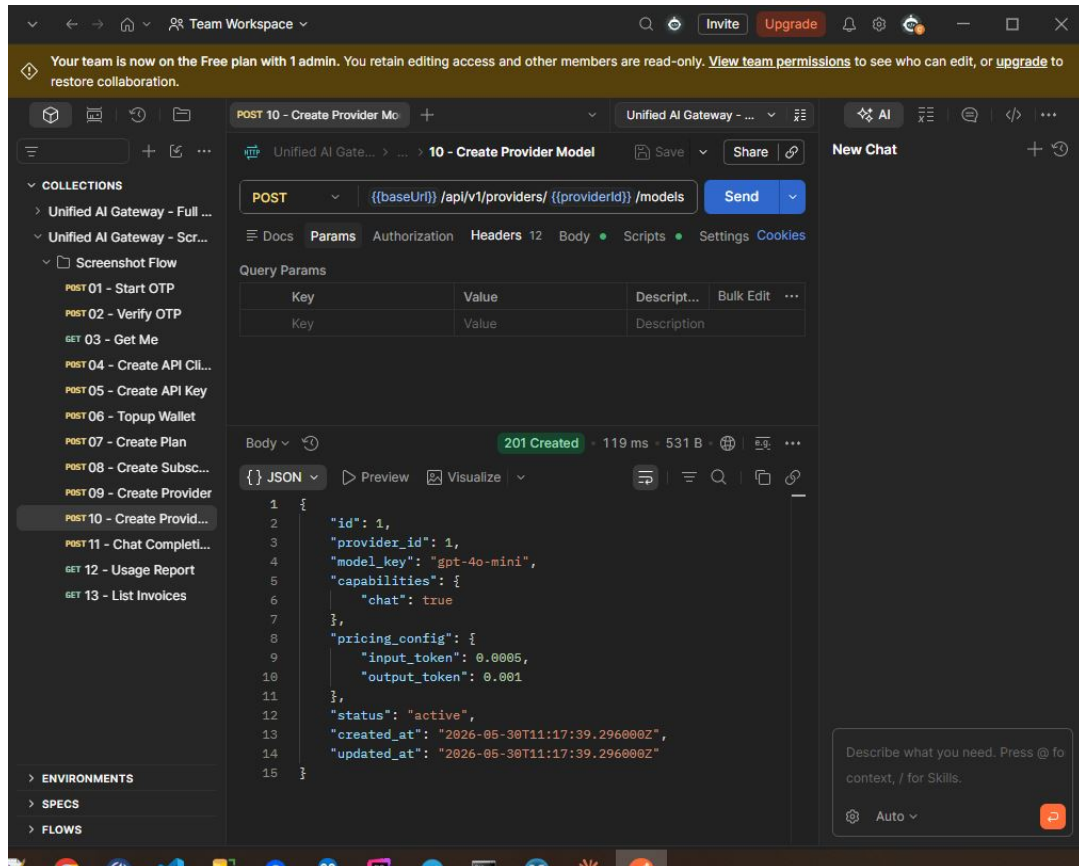
هر دوی این ارائه‌دهندگان با فرمت OpenAI سازگار هستند؛ آداپتور OpenAI-Compatible تنها Base URL و کلید API آن‌ها را جایگزین می‌کند.

برای فعال‌سازی هر ارائه‌دهنده در سامانه باید ابتدا رکورد آن در پایگاه داده ثبت شود. شکل ۲-۴ فرآیند ثبت ارائه‌دهنده‌ی Groq را نشان می‌دهد.



شکل ۲-۴: ثبت ارائه‌دهنده‌ی Groq در سامانه

پس از ثبت ارائه‌دهنده، مدل‌های قابل استفاده‌ی آن نیز باید تعریف شوند. شکل ۴-۳ ثبت مدل llama-3.3-70b-versatile برای ارائه‌دهنده‌ی Groq را نشان می‌دهد.



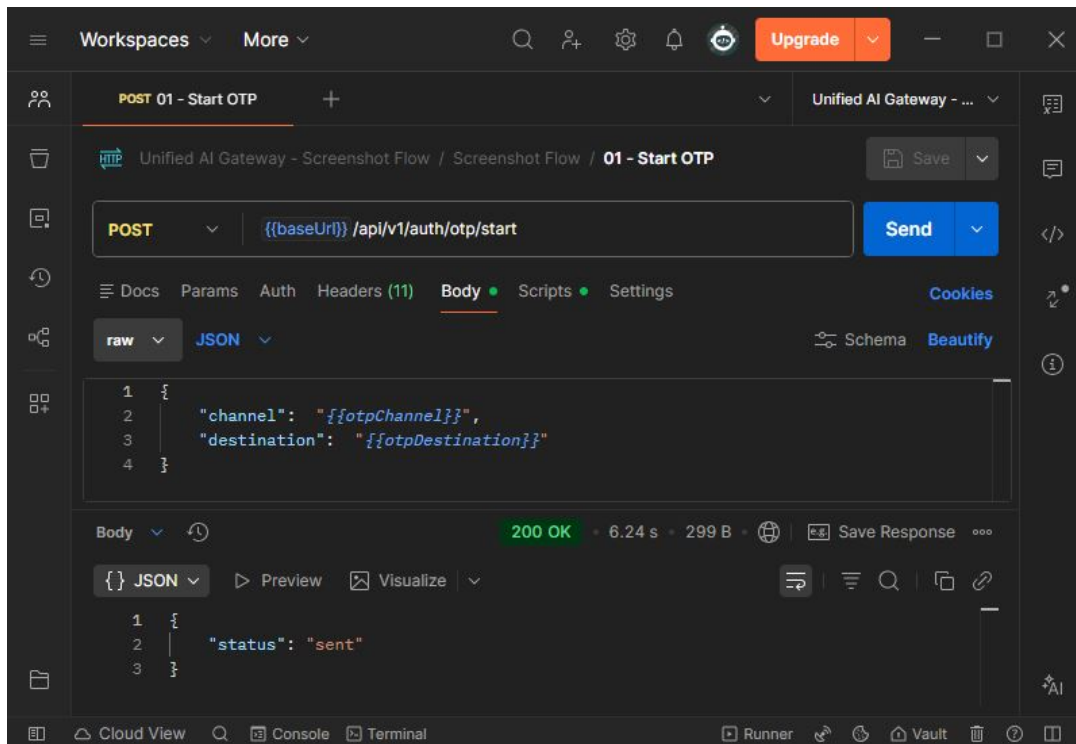
شکل ۴-۳: ثبت مدل llama-3.3-70b-versatile برای ارائه‌دهنده‌ی Groq

۶-۴ پیاده‌سازی احراز هویت OTP

جریان احراز هویت سامانه مبتنی بر رمز یک‌بارمصرف^۲ است. کاربر با شماره‌ی تلفن یا ایمیل خود درخواست ورود می‌دهد، سامانه یک رمز شش‌رقمی تولید و ارسال می‌کند، و کاربر پس از دریافت، آن را برای تأیید هویت ارسال می‌نماید. در صورت تأیید، یک توکن Sanctum صادر می‌شود.

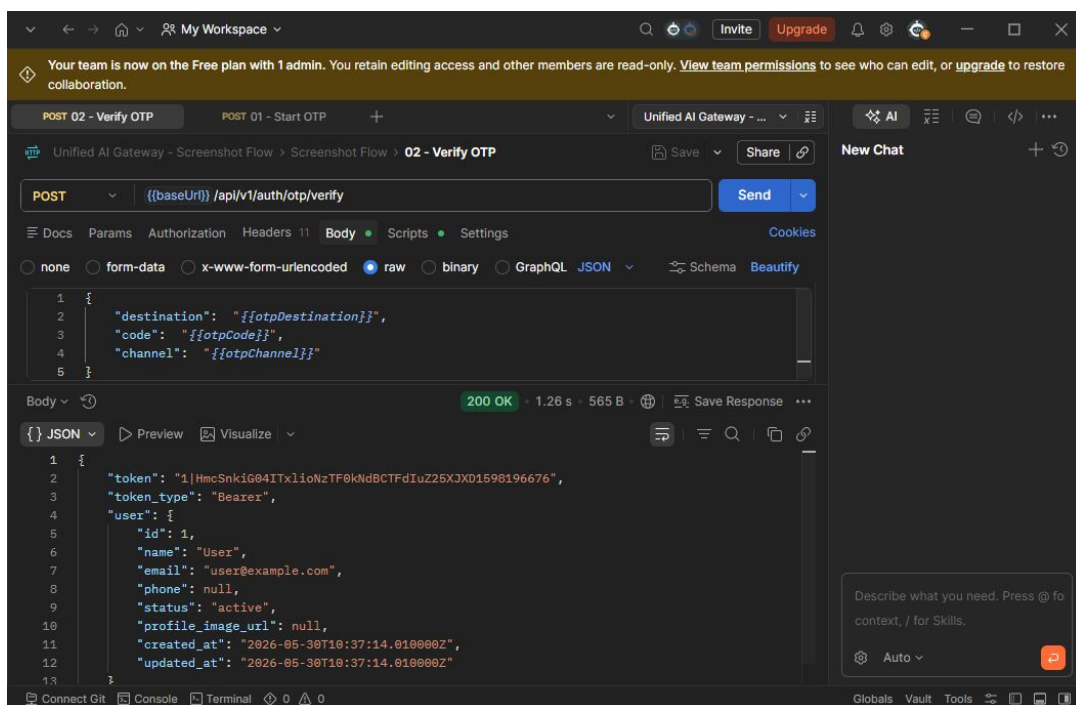
شکل ۴-۴ فراخوانی نقطه‌ی پایانی ارسال رمز یک‌بارمصرف را نشان می‌دهد.

^۲One-Time Password (OTP)



شکل ۴-۴: ارسال درخواست دریافت رمز یک‌بارمصرف

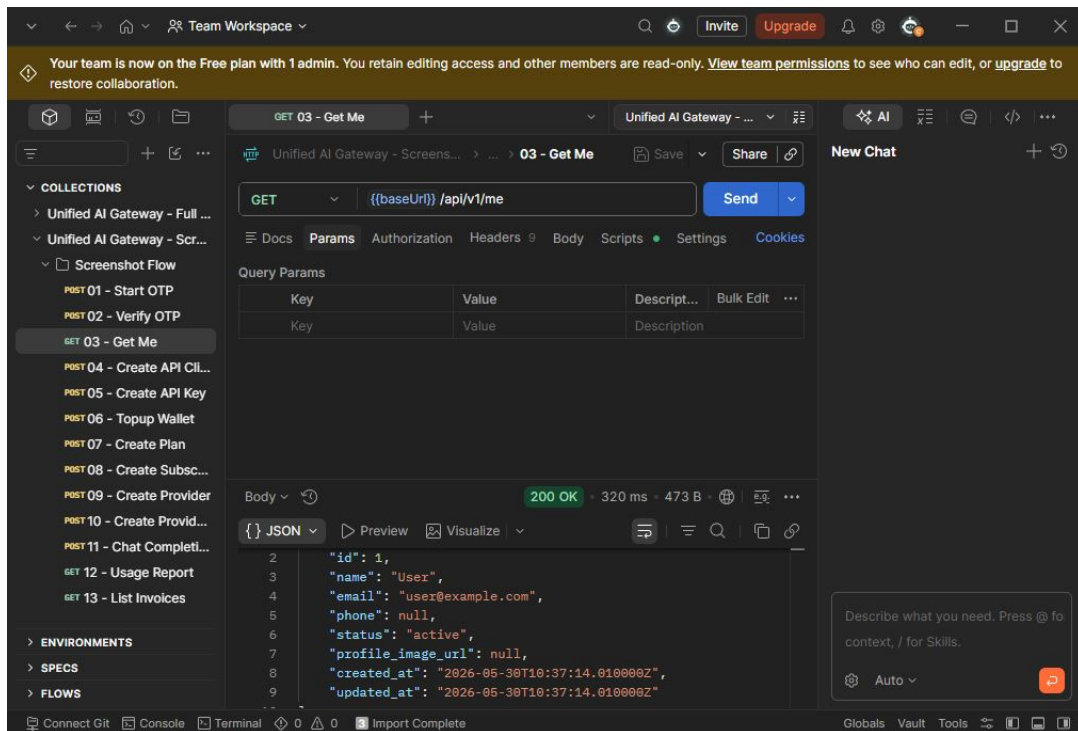
پس از دریافت رمز، کاربر آن را به نقطه‌ی پایانی تأیید ارسال می‌کند و در ازای آن توکن احراز هویت دریافت می‌نماید. شکل ۴-۵ این مرحله را نشان می‌دهد.



شکل ۴-۵: تأیید رمز یک‌بارمصرف و دریافت توکن احراز هویت

پس از احراز هویت موفق، کاربر با ارسال توکن در هدر Authorization می‌تواند اطلاعات

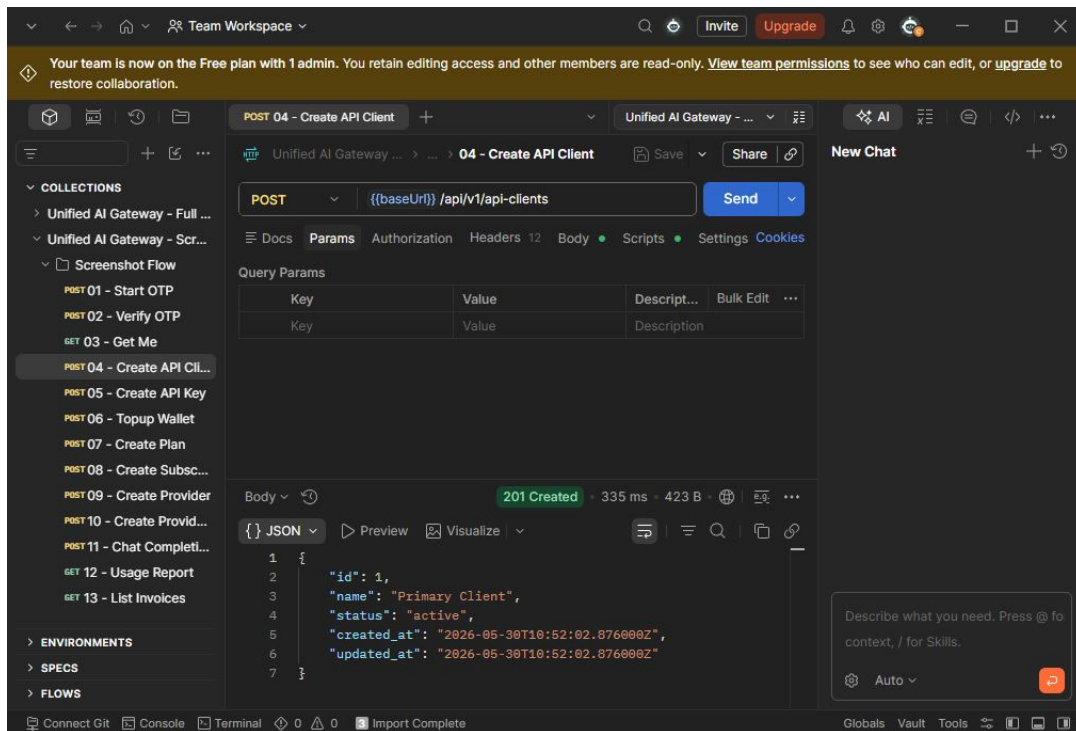
پرو فایل خود را دریافت کند. شکل ۴-۶ نتیجه‌ی این فراخوانی را نشان می‌دهد.



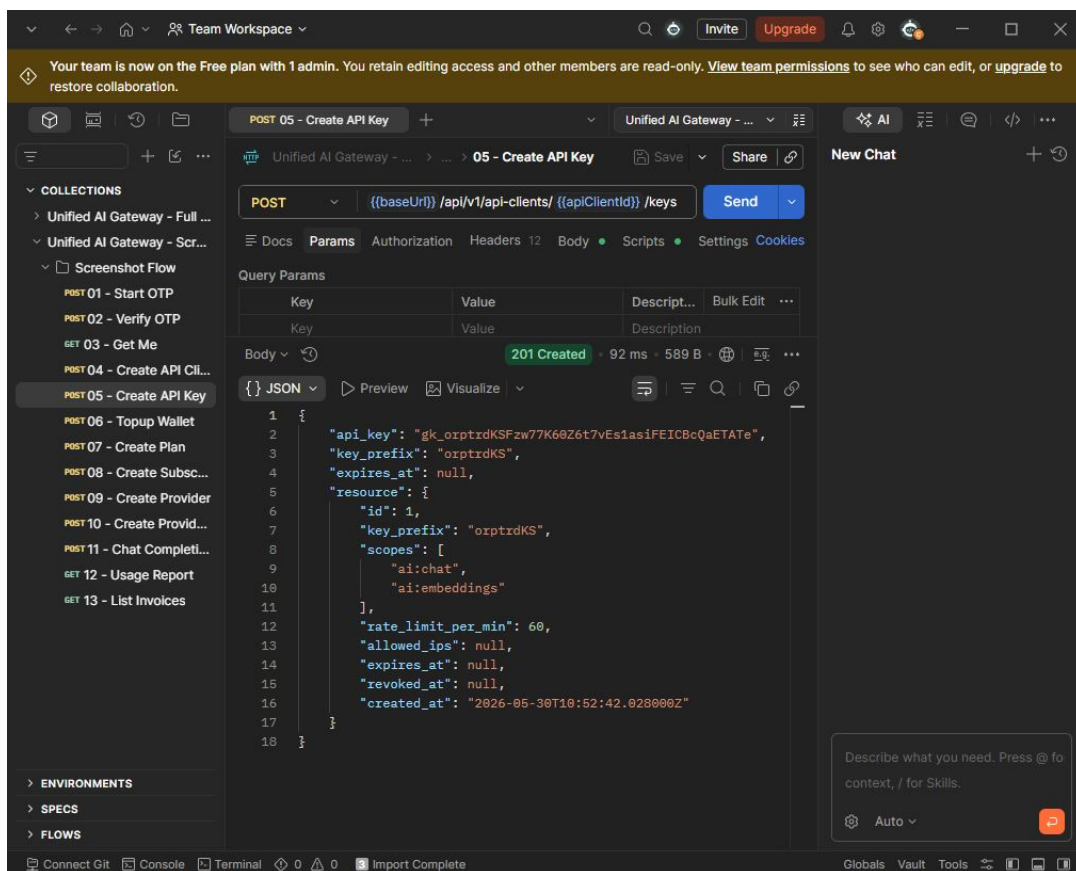
شکل ۴-۶: دریافت اطلاعات پرو فایل کاربر پس از احراز هویت

۷-۴ پیاده‌سازی مدیریت کلیدهای API

برای دسترسی برنامه‌ای به نقاط پایانی سامانه‌ی واسط، هر کاربر باید ابتدا یک کلاینت API^۳ ایجاد کند و سپس زیر آن کلید API بسازد. کلید تنها یک‌بار—در زمان صدور—نمایش داده می‌شود و تنها هش آن در پایگاه داده ذخیره می‌شود.



شکل ۷-۴: ایجاد کلاینت API برای یک کاربر

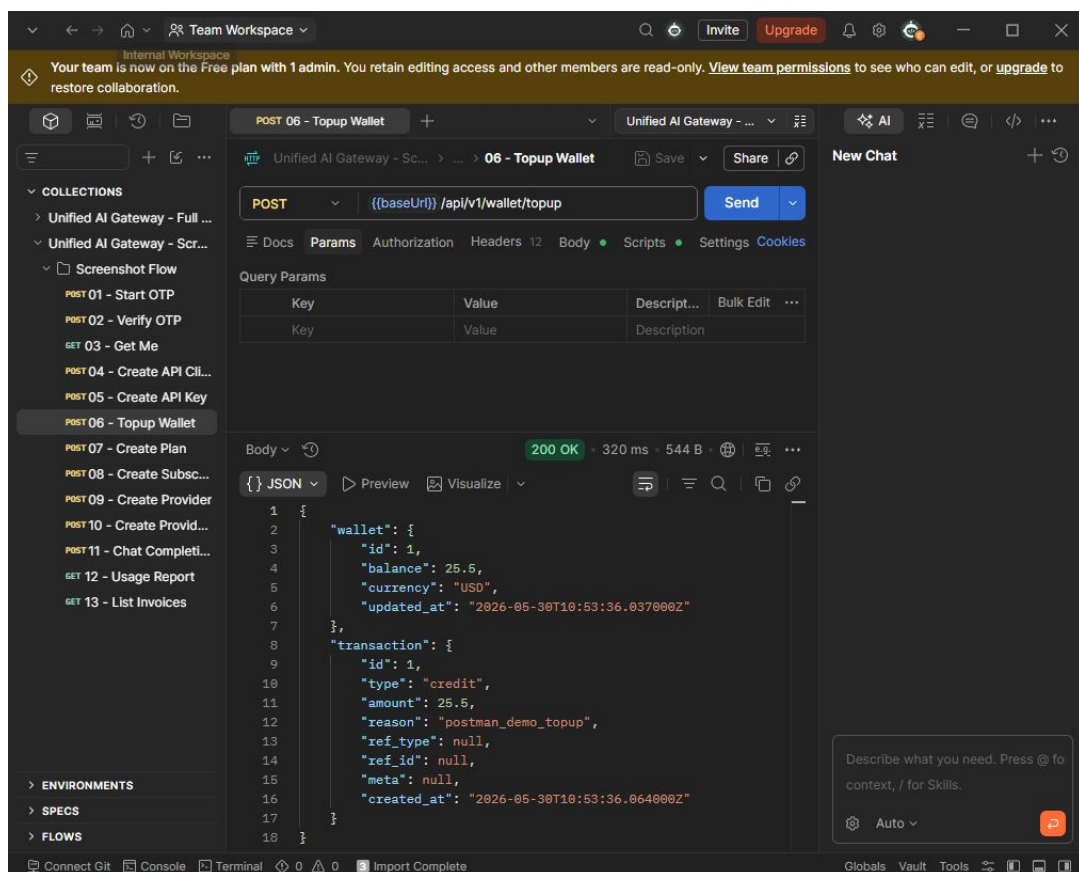


شکل ۸-۴: ایجاد کلید API و دریافت توکن دسترسی

۸-۴ پیاده‌سازی سیستم صورت‌حساب

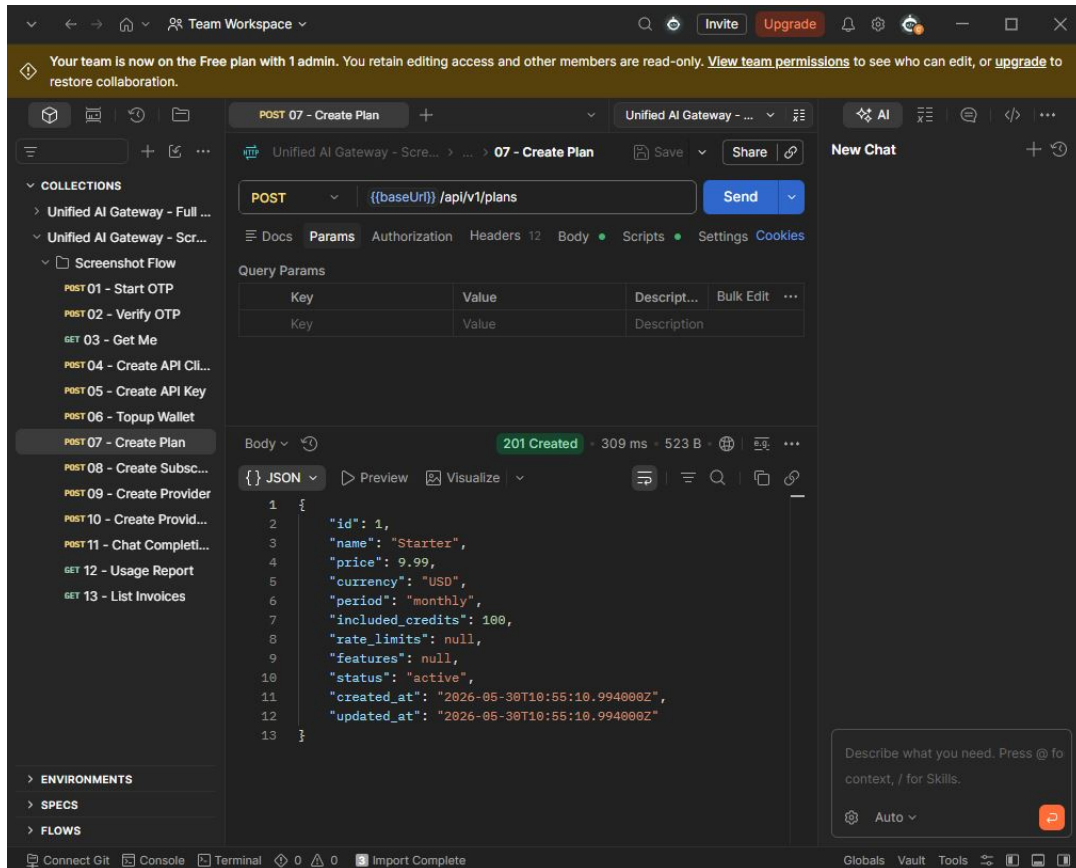
سیستم صورت‌حساب شامل سه بخش اصلی است: کیف پول کاربری، مدیریت طرح‌های اشتراک و ثبت مصرف. کاربران می‌توانند کیف پول خود را شارژ کنند، اشتراک بخرند یا به صورت مستقیم از موجودی کیف پول پرداخت نمایند.

شکل ۴-۹ فرآیند شارژ کیف پول را نشان می‌دهد.



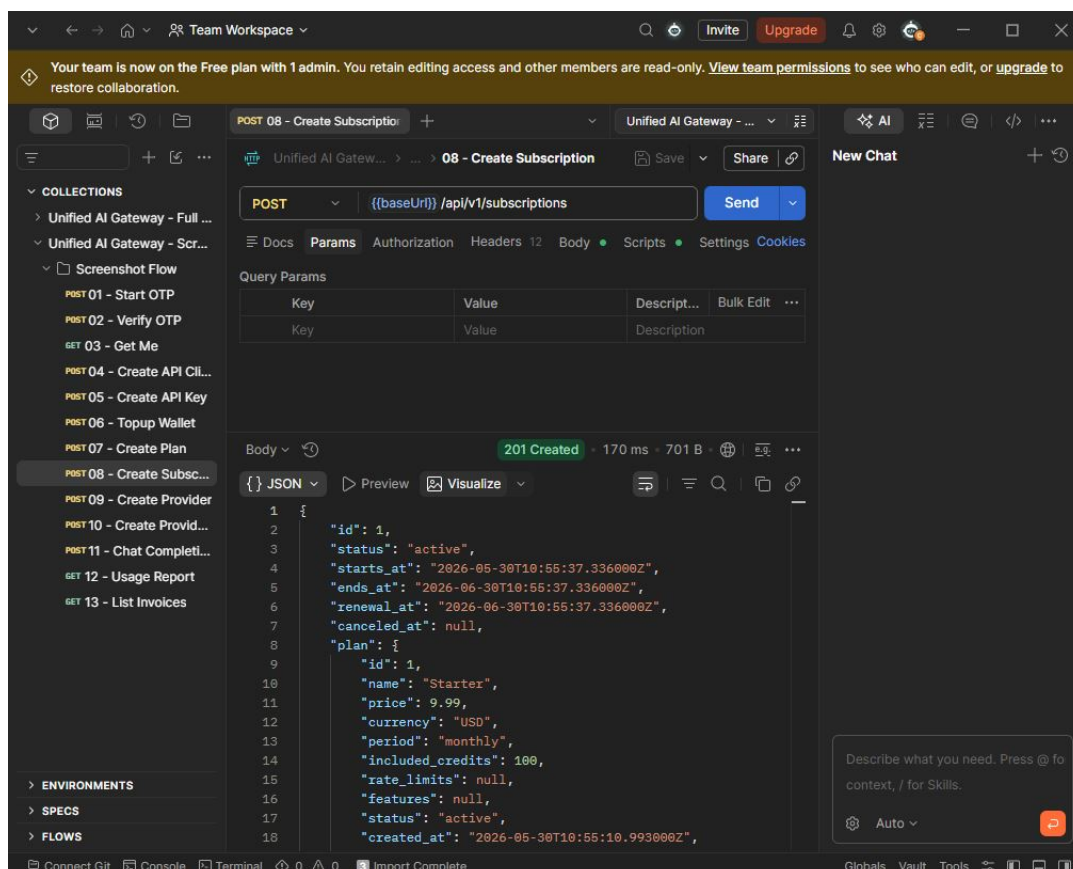
شکل ۴-۹: شارژ کیف پول کاربری

برای مدیریت اشتراک‌ها، ابتدا طرح اشتراک توسط مدیر تعریف می‌شود. هر طرح شامل سقف توکن ماهانه، قیمت و بازه‌ی زمانی است. شکل ۴-۱۰ فرآیند ایجاد یک طرح اشتراک را نشان می‌دهد.



شکل ۴-۱۰: ایجاد طرح اشتراک

پس از تعریف طرح، کاربران می‌توانند آن را خریداری کنند. با فعال شدن اشتراک، مصرف از سقف اشتراک کسر می‌شود و تنها در صورت تجاوز از سقف، از کیف پول برداشت می‌شود. شکل ۴-۱۱ این فرآیند را نشان می‌دهد.

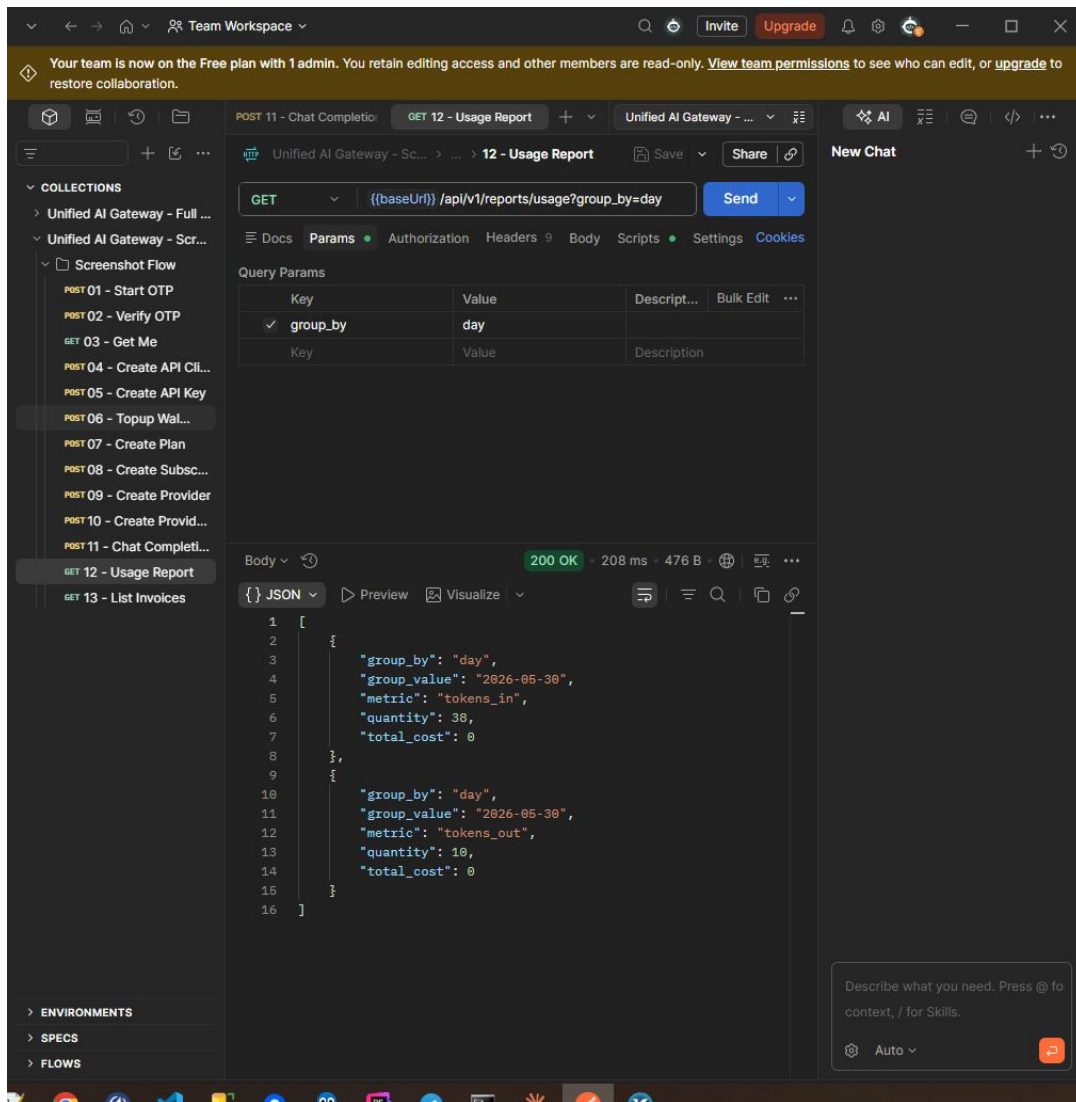


شکل ۴-۱۱: خرید اشتراک توسط کاربر

۹-۴ پیاده‌سازی گزارش‌دهی و ارسال گزارش خارجی

سامانه گزارش‌های تفصیلی مصرف را برای هر کاربر و به تفکیک ارائه‌دهنده ارائه می‌دهد. این گزارش‌ها شامل تعداد توکن‌های ورودی و خروجی، هزینه، تأخیر پاسخ و وضعیت هر درخواست هستند. علاوه بر API داخلی، سامانه می‌تواند گزارش‌ها را به سرویس‌های مدیریت گزارش خارجی مانند Loki یا BetterStack نیز ارسال کند.

شکل ۴-۱۲ نمونه‌ای از گزارش مصرف کاربر را نشان می‌دهد.



شکل ۴-۱۲: گزارش مصرف کاربر به تفکیک درخواست

۱۰-۴ داشبورد Playground

داشبورد Playground یک رابط کاربری تحت وب است که با Vue.js پیاده‌سازی شده و امکان آزمایش تمامی نقاط پایانی سامانه را فراهم می‌کند. کاربر می‌تواند ارائه‌دهنده، مدل و کلید API را انتخاب کرده و درخواست را مستقیماً از مرورگر ارسال نماید.

فصل ۵

ارزیابی

۱-۵ مقدمه

۲-۵ ارزیابی فنی

۱-۲-۵ آزمون‌های واحد

۲-۲-۵ آزمون‌های ویژگی

۳-۲-۵ نتایج آزمون

۳-۵ ارزیابی عملکرد

۱-۳-۵ تأخیر پاسخ

۲-۳-۵ دقت صورت حساب‌گذاری

۴-۵ مقایسه با اهداف

فصل ۶

نتیجه‌گیری

۱-۶ جمع‌بندی

۲-۶ دستاوردها

۳-۶ محدودیت‌ها

۴-۶ کارهای آتی

کتابنامه

- <https://platform.openai.com/docs> documentation. platform OpenAI OpenAI. [۱]
.۲۹-۰۱-۲۰۲۵ Accessed: .۲۰۲۴
- <https://console.groq.com/docs> documentation. API GroqCloud Groq. [۲]
.۲۹-۰۱-۲۰۲۵ Accessed: .۲۰۲۴
- <https://openrouter.ai/docs> platform. API AI Unified OpenRouter. [۳]
.۲۹-۰۱-۲۰۲۵ Accessed: .۲۰۲۴

مستندات API

این پیوست فهرست نقاط پایانی اصلی سامانه را به همراه نمونه‌ی درخواست و پاسخ ارائه می‌دهد. تمامی نقاط پایانی زیر Base URL زیر قرار دارند:

`http://localhost/api/v1`

درخواست‌های احراز هویت داشبورد با هدر `Authorization: Bearer <token>` و درخواست‌های سامانه‌ی واسط با هدر `Authorization: Bearer <api-key>` ارسال می‌شوند.

۱- نقاط پایانی احراز هویت

POST /auth/otp/start

درخواست ارسال رمز یک‌بارمصرف. فیلد `destination` شماره تلفن یا ایمیل و فیلد `channel` مقدار `sms` یا `email` را می‌پذیرد. نمونه‌ی این فراخوانی در شکل ۴-۴ (صفحه ۱۴) نمایش داده شده است.

POST /auth/otp/verify

تأیید رمز یک‌بارمصرف. در صورت موفقیت، توکن احراز هویت `Sanctum` را برمی‌گرداند. نمونه در شکل ۴-۵ (صفحه ۱۴) قابل مشاهده است.

۲- نقاط پایانی کاربر

GET /user/me

اطلاعات کاربر جاری را برمی‌گرداند. نیاز به توکن احراز هویت دارد. نمونه در شکل ۴-۶ (صفحه ۱۵) آمده است.

۳- نقاط پایانی کلیدهای API

POST /clients

ایجاد کلاینت API جدید. شکل ۴-۷ (صفحه ۱۶) نمونه را نشان می‌دهد.

POST /clients/{client}/keys

صدور کلید API زیر یک کلاینت. کلید تنها یک‌بار در پاسخ این درخواست نمایش داده می‌شود و در پایگاه داده تنها هشت آن ذخیره می‌گردد. نمونه در شکل ۴-۸ (صفحه ۱۶) آمده است.

۴- نقاط پایانی هوش مصنوعی

POST /ai/chat/completions

نقطه‌ی پایانی تکمیل گفتگو، سازگار با فرمت OpenAI. فیلدهای اجباری model، provider و messages هستند. نمونه‌ی کامل در شکل ۴-۱ (صفحه ۱۱) آمده است.

POST /ai/embeddings

تبدیل متن به بردار جاسازی^۱.

POST /ai/images/generations

تولید تصویر از روی متن.

POST /ai/audio/transcriptions

تبدیل فایل صوتی به متن.

POST /ai/audio/speech

تبدیل متن به فایل صوتی.

^۱Embedding

۵- نقاط پایانی کیف پول و اشتراک

POST /wallet/topup

شارژ کیف پول کاربر. نمونه در شکل ۴-۹ (صفحه ۱۷) نشان داده شده است.

POST /plans

ایجاد طرح اشتراک جدید. نمونه در شکل ۴-۱۰ (صفحه ۱۸) قابل مشاهده است.

POST /subscriptions

خرید اشتراک. نمونه در شکل ۴-۱۱ (صفحه ۱۹) آمده است.

۶- نقاط پایانی گزارش‌ها

GET /reports/usage

گزارش تفصیلی مصرف کاربر جاری. پاسخ شامل تعداد توکن، هزینه، تأخیر و وضعیت هر درخواست است. نمونه در شکل ۴-۱۲ (صفحه ۲۰) نشان داده شده است.